Object Detection with CNNs

R-CNN → Fast R-CNN → Faster R-CNN

My study notes on the region-based object detection family
Deep Learning Roadmap | CNN Topic — Object Detection

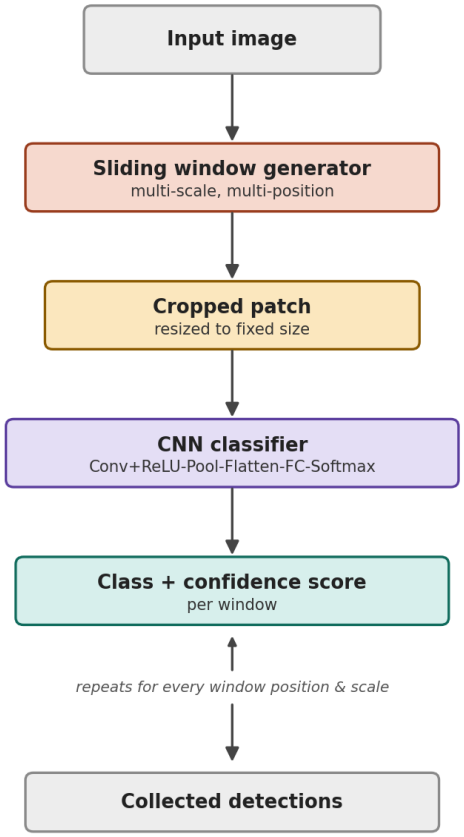
These notes cover why plain CNN classifiers aren't enough for detection, how the sliding window idea works and why it's slow, and then how R-CNN, Fast R-CNN, and Faster R-CNN each fix the problem of the previous one, step by step.

0. Why a normal CNN classifier is not enough

A normal CNN (Conv+ReLU → Pool → Conv+ReLU → Pool → Flatten+FC → Softmax) can only look at one whole image and say what single thing it thinks is in it. It cannot say *where* the object is, and it cannot handle an image that has multiple different objects in it at once. Object detection needs both the class AND the location (a bounding box) for every object in the image. This is the problem the following architectures try to solve.

1. Sliding Window — the first (naive) idea

The simplest fix: if the CNN can only classify one thing, just chop the image into lots of small pieces and classify every piece separately. A small window slides across the image at every position, and at multiple scales (sizes), because we don't know in advance how big the object will look in the image.



Sliding window pipeline

How the flow works, step by step

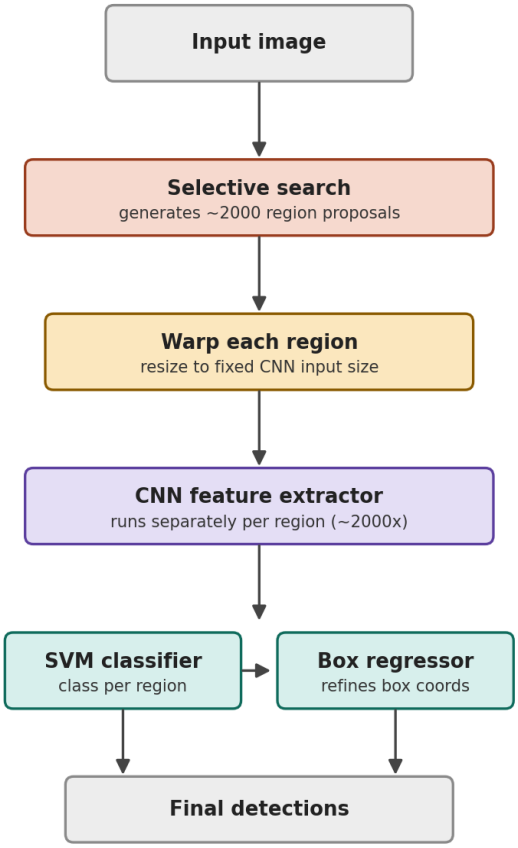
- **Sliding window generator** scans the image at every position and at multiple scales. A small window catches objects that look small in the image (near-small things, or far-away things). A big window catches objects that look big in the image.

- **Cropped patch**: at every window position, whatever pixels are currently under the window get sliced out — this happens blindly, no matter what is actually inside (could be empty background). The crop is then resized to the CNN's fixed input size.
- **CNN classifier**: the exact same CNN structure as before (Conv+ReLU → Pool → ... → Softmax) looks at just that one cropped patch.
- **Class + confidence score**: the Softmax gives a probability for every possible class for that ONE patch. The class with the highest probability is picked as the predicted class, and that probability is the confidence. Example for one patch: elephant 0.03, lion 0.02, mouse 0.92, background 0.03 → class = mouse, confidence = 0.92.
- This crop → classify → score cycle **loops** for every window position and every scale.
- **Collected detections**: after the loop finishes, we have scores for every patch — most are background, a few are actual objects. Those are the final detections.

Problem with this approach: if the image is scanned at many positions and many scales, the CNN has to run thousands of times on one single image — most of those runs are wasted on empty background. This is very slow. This exact problem is what R-CNN tries to fix.

2. R-CNN (Regions with CNN features)

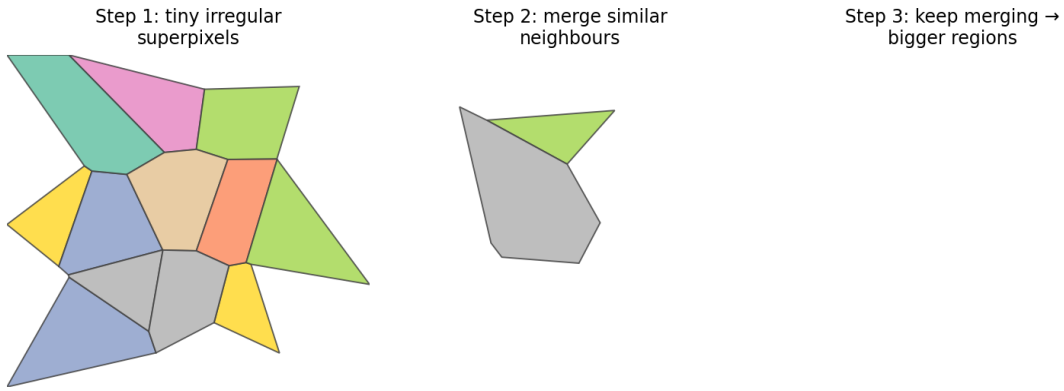
Instead of blindly scanning every window position, R-CNN's idea is: don't scan blindly, first *propose* a smart, small set of regions that are actually likely to contain an object, and only run the CNN on those.



R-CNN pipeline

2.1 Selective Search

Selective search is the algorithm that generates the region proposals (~2000 per image). It does not know anything about what objects look like — it only works with raw pixel colour and texture values (numbers), and finds regions purely through similarity.



How selective search grows regions, left to right

- Every pixel is just a set of numbers (RGB colour values) — there is no understanding of what the image actually contains.
- Start by treating every tiny pixel cluster as its own separate mini-region. These are called **superpixels**, and they are irregular shapes (not a fixed grid like 4×4 blocks) — they follow the natural colour/texture boundaries in the image, for example the curved edge of an elephant's ear.
- Compare each region to its neighbouring region's colour and texture values. If the values are close/similar → merge them into one bigger region. If very different → keep them separate (that jump usually becomes a natural object boundary).
- Repeat this compare-and-merge process again and again, growing bigger regions each round.
- **Every single stage** of merging (small groups, medium groups, big groups) becomes a candidate region proposal — not just the final biggest group. This is why the final ~2000 proposals come in many different sizes.

Region similarity formula

```
s(r_i, r_j) = a1·s_color + a2·s_texture + a3·s_size + a4·s_fill
```

Color similarity, using colour histogram bins c_i^k and c_j^k for each region:

```
s_color(r_i, r_j) = sum over k of min(c_i^k, c_j^k)
```

In plain words: split each region's colours into a few buckets (bins). For each bin, take the smaller value between the two regions, then add all of these up. If both regions share a lot of the same colour, the overlap score is high → more likely to merge.

Worked example: Region 1 = [0.5, 0.3, 0.2], Region 2 = [0.4, 0.4, 0.2] (percentages across 3 colour bins). s_color = min(0.5,0.4) + min(0.3,0.4) + min(0.2,0.2) = 0.4 + 0.3 + 0.2 = **0.9** (high overlap → likely merge).

2.2 Warping

Selective search's ~2000 proposals are all different sizes and shapes (since they came from irregular merging, not a grid). But the CNN needs one fixed input size. So each region is **warped** — stretched/resized in raw pixel space — to match the CNN's fixed input size, e.g. 224×224.

```
s_x = W_target / W_region    s_y = H_target / H_region
```

2.3 CNN Feature Extractor

This is the same CNN structure from before: Conv+ReLU → Pool → Conv+ReLU → Pool → Flatten → FC. But here it **stops right before Softmax** — there is no classification happening inside the CNN. Its only job is to output a **feature vector** (a list of numbers describing that region). The CNN runs **separately, once per region** — about 2000 times per image. This repeated, overlapping computation is the main reason R-CNN is slow.

2.4 SVM Classifier

SVM = **Support Vector Machine**, a classic machine learning algorithm (not a neural network, and it existed before CNNs became popular for vision). It is a supervised classifier: given labelled training data, it learns the best boundary (a hyperplane) that separates classes with the maximum possible margin (gap) between them. In R-CNN, since the CNN only outputs a feature vector (no Softmax), a separate SVM takes that feature vector and decides the class.

```
f(x) = w · x + b    (decision function, sign of f(x) gives the class)
```

Where x = feature vector from the CNN, w = learned weight vector, b = learned bias. Training objective (maximise the margin):

```
minimise (1/2)||w||^2, subject to y_i(w·x_i + b) ≥ 1 for every training
sample i
```

2.5 Box Regressor

Selective search's box for a region is only a rough guess. The box regressor does not invent a new box from scratch — it takes that existing rough box as input and **adjusts** its coordinates (x, y, width, height) so it fits more tightly around the actual object.

Let P = (P_x, P_y, P_w, P_h) be the proposal box and G = (G_x, G_y, G_w, G_h) be the ground-truth box. The regressor is trained to predict these targets:

```
t_x = (G_x - P_x)/P_w    t_y = (G_y - P_y)/P_h    t_w = log(G_w/P_w)    t_h =
log(G_h/P_h)
```

Then the refined box is reconstructed from the predicted d_x, d_y, d_w, d_h as:

```
G_hat_x = P_w·d_x + P_x    G_hat_y = P_h·d_y + P_y    G_hat_w = P_w·e^d_w
G_hat_h = P_h·e^d_h
```

Worked numeric example (one region, e.g. the elephant)

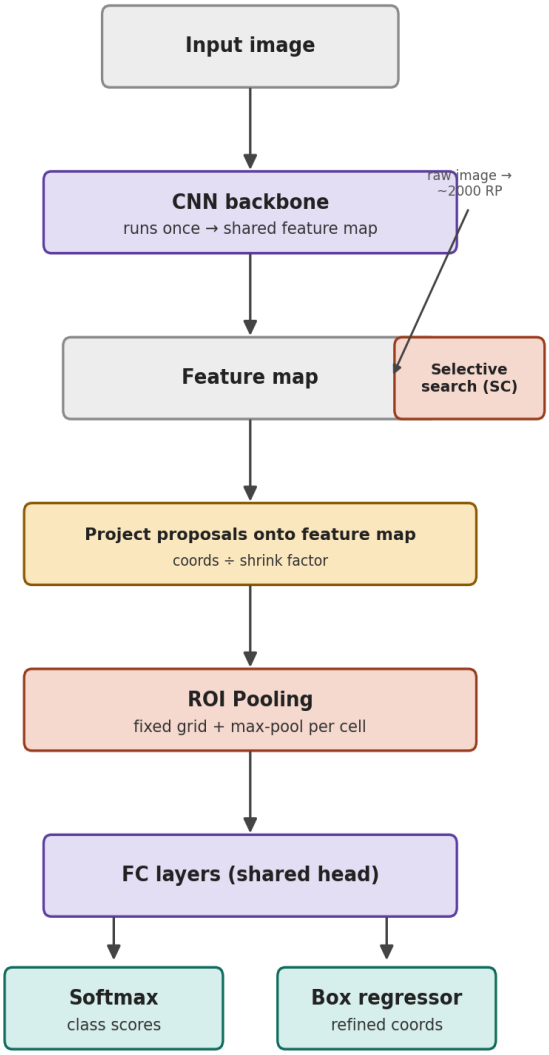
Proposal box P = (50, 50, 150, 200). Ground truth G = (60, 55, 140, 190).
t_x = (60-50)/150 = 0.067 t_y = (55-50)/200 = 0.025 t_w = log(140/150) = -0.069 t_h = log(190/200) = -0.051
Say the trained regressor predicts values close to these targets: d_x=0.07, d_y=0.02, d_w=-0.07, d_h=-0.05.
Refined box: G_hat_x = 150(0.07)+50 = 60.5, G_hat_y = 200(0.02)+50 = 54, G_hat_w = 150·e^-0.07 = 139.9, G_hat_h = 200·e^-0.05 = 190.2
Final refined box (60.5, 54, 139.9, 190.2) is very close to the ground truth (60, 55, 140, 190) — the regressor successfully tightened the box.

R-CNN's core weakness

The CNN runs separately, from scratch, about 2000 times per image — on regions that heavily overlap each other. This repeated computation is very expensive and slow. Fast R-CNN's whole idea is to fix exactly this.

3. Fast R-CNN

Key question Fast R-CNN asks: since the ~2000 regions overlap so much, why run the CNN once per region? Just run the CNN **once** on the whole image, and reuse that single computation for every region.



Fast R-CNN pipeline

What actually changed from R-CNN

	R-CNN	Fast R-CNN
CNN runs	~2000x per image	1x per image
Region extraction	Warp raw pixels, then CNN	ROI pooling on shared feature map
Classifier	Separate SVM	Softmax (same network)

Box regressor	Separate model	Same network, trained jointly
Region proposals	Selective search	Selective search (unchanged)

3.1 Two parallel, independent paths

The image goes down two separate paths that do not depend on each other: **Path A:** image → CNN backbone → one shared feature map. **Path B:** image → Selective search → ~2000 region proposals (just box coordinates on the original image). Selective search does not need the CNN's output to run — both paths only meet again in the next step.

3.2 Projecting proposals onto the feature map

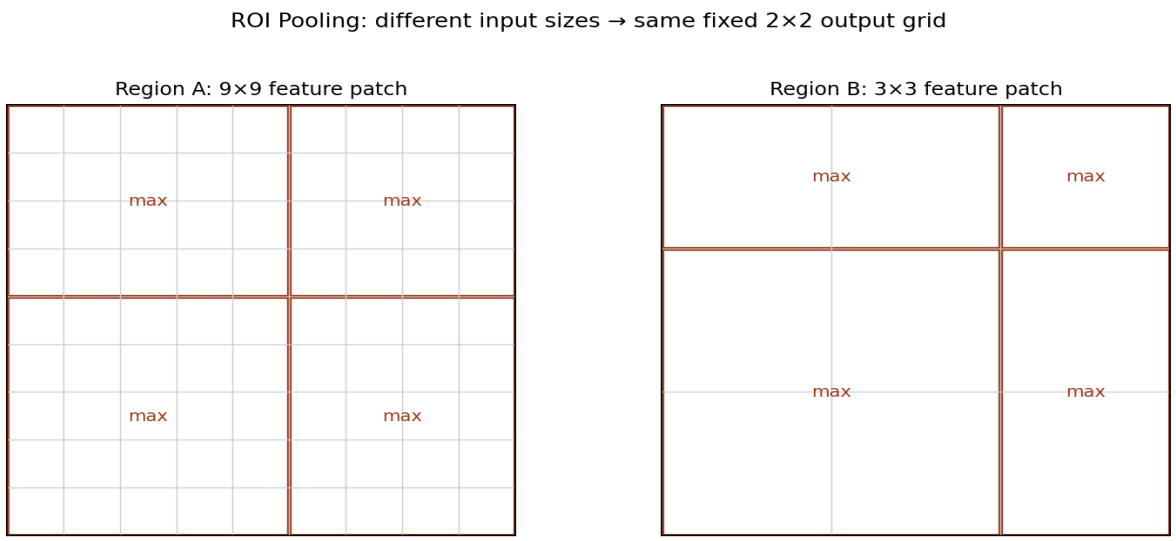
The CNN backbone shrinks the image as it passes through conv and pooling layers. For example, a 224×224 image might become a 14×14 feature map — a 16× shrink. Selective search's boxes are given in original image coordinates, so before ROI pooling can use them, they must be scaled down by the same shrink factor to correctly point at the matching small patch on the feature map.

```
x_feat = x_image / shrink_factor    y_feat = y_image / shrink_factor
w_feat = w_image / shrink_factor    h_feat = h_image / shrink_factor
```

Example: a box at (80, 80, 128, 128) on a 224×224 image, with shrink factor 16, becomes (5, 5, 8, 8) on the 14×14 feature map.

3.3 ROI Pooling

Each projected region is now a small patch of the feature map, but every region has a **different size**. The FC layers need one fixed input size. ROI Pooling solves this without warping/resizing pixels — it **divides** the patch into a fixed grid (e.g. 7×7) and takes the **maximum value inside each grid cell** (max-pooling). No matter the input size, this always produces a fixed 7×7 output.



Two different-sized regions (9x9 and 3x3) both end up as a 2x2 output after ROI pooling, just as any size would map to a fixed 7x7 grid in practice

When the region is bigger than the grid (like 9x9 → 2x2), some grid cells cover more pixels. When the region is smaller than the grid (like 3x3 → 2x2), some source pixels get reused/overlap across cells. Either way, the max value of whatever falls inside each cell becomes that cell's single output number — giving exactly 2x2 = 4 numbers (or 7x7 = 49 numbers) every single time.

3.4 FC layers (shared head) and outputs

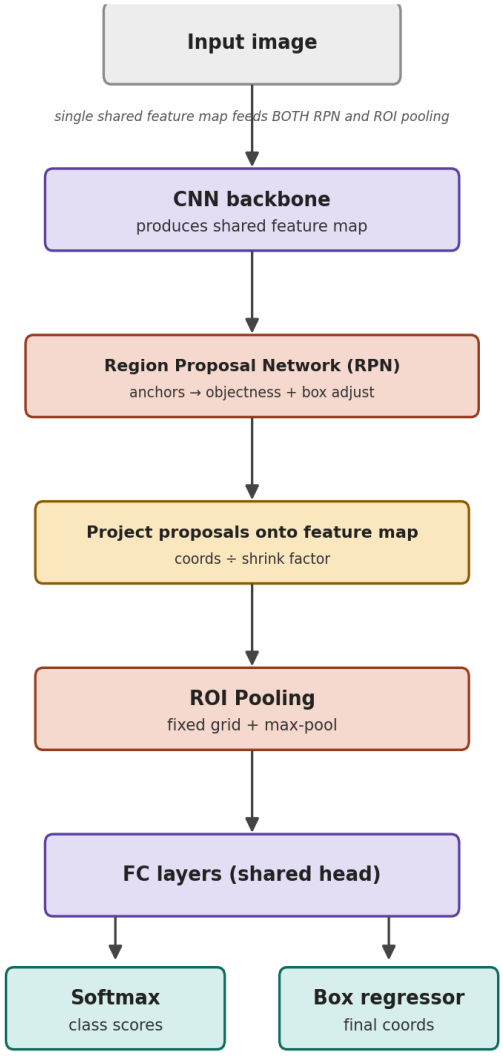
The fixed-size pooled output is flattened into a 1D vector and passed through shared fully connected layers — "shared" meaning the same FC weights are reused for every region, not a separate network per region. The FC output then splits into two branches, trained together end-to-end: **Softmax** gives the class scores, and the **box regressor** gives the refined box coordinates (same regression idea as in R-CNN).

Fast R-CNN's remaining weakness

Selective search is still a separate, slow, non-learnable algorithm that runs on the CPU. Now that everything else is fast, selective search becomes the new bottleneck. Faster R-CNN's whole idea is to replace it with a trainable neural network.

4. Faster R-CNN

Key question Faster R-CNN asks: selective search is the last non-neural, slow piece in the pipeline — why not replace it with a small neural network too, so the whole model is trainable end-to-end and runs on the GPU?



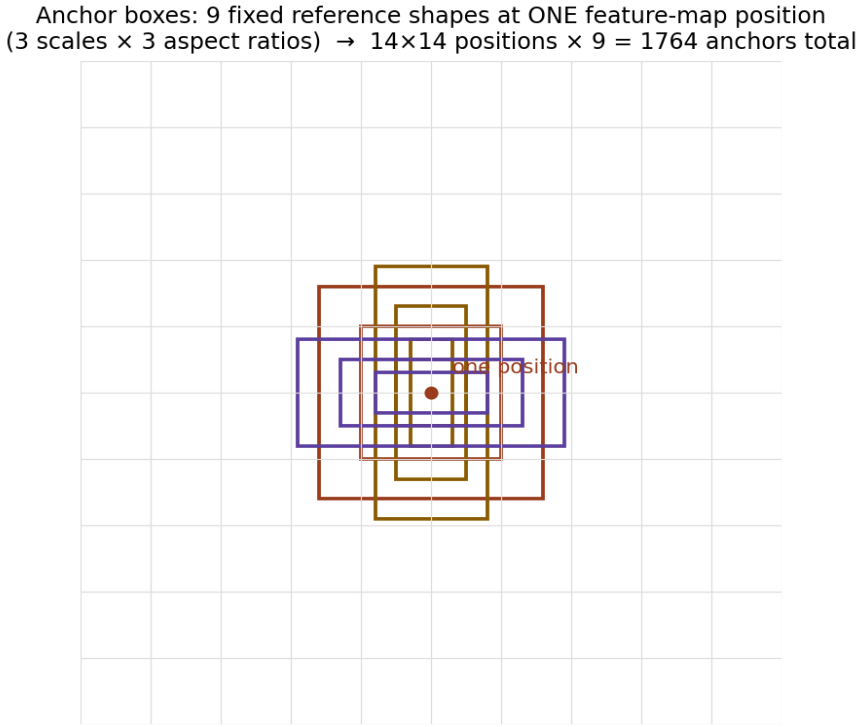
Faster R-CNN pipeline

What actually changed from Fast R-CNN

- **Selective search is completely removed.** Region proposals now come from a **Region Proposal Network (RPN)** — a small neural network.
- There is **only one path now**, not two parallel paths. The image goes through the CNN backbone once → produces a feature map → the RPN reuses that **same** feature map directly to generate proposals. No separate selective-search path is needed.
- Everything after proposal generation stays the same as Fast R-CNN: project proposals onto the feature map, ROI pooling, shared FC layers, Softmax + box regressor.

4.1 Anchor boxes

The RPN does not scan raw pixels like selective search did. Instead, at **every position** on the feature map, it places a fixed set of pre-made reference box shapes called **anchors** — typically 9 per position (3 scales: small/medium/large, × 3 aspect ratios: tall/square/wide). These anchors are fixed in advance and are not learned or moved — think of them as a grid of "guess boxes" laid down everywhere before any prediction happens.

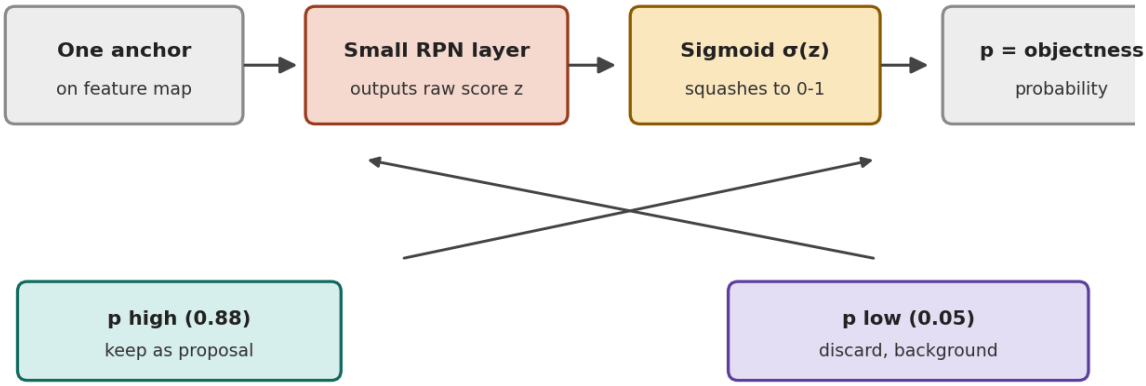


9 anchor boxes at one feature-map position

For a 14×14 feature map: 14 × 14 × 9 = **1764 anchor boxes** in total for one image. Reason for anchors: predicting a box's exact size and position from nothing is a hard problem. But if a roughly-shaped box is already sitting there, the network just needs to answer "is there an object here, yes or no" and "nudge this box a little" — a much easier learning problem.

4.2 Objectness score

For every one of the 1764 anchors, the RPN asks one simple question: does this anchor likely contain *some* object (any object, not which class) or is it just background?



Objectness score flow for one anchor

```
p = sigma(z)    where sigma(z) = 1 / (1 + e^-z)
```

z is the raw score (logit) from a small RPN layer for that anchor, and sigma squashes it into a clean probability p between 0 and 1.
Example: z = 2.0 → p = 1/(1+e^-2.0) = 1/(1+0.135) = **0.88** → 88% likely an object → keep as proposal.
Example: z = -3.0 → p = 1/(1+e^3.0) = 1/(1+20.09) = **0.047** → only 4.7% → discard, background.

4.3 Labelling anchors during training — IoU

To train the RPN, every anchor needs a label (object or not). This is decided by comparing the anchor box A to the nearest ground-truth box G using **Intersection over Union (IoU)**:

```
IoU(A, G) = Area(A ∩ G) / Area(A ∪ G)
```

Worked example: anchor area = 100, ground truth area = 120, overlap area = 80. Union = 100 + 120 - 80 = 140. IoU = 80/140 = **0.57**.
Labelling rule: IoU ≥ 0.7 → positive (object, y=1). IoU ≤ 0.3 → negative (background, y=0). Between 0.3 and 0.7 → ignored during training (too ambiguous).

4.4 Box adjustment for anchors

Same regression idea as R-CNN/Fast R-CNN's box regressor, but applied to the anchor box instead of a selective-search box:

```
t_x = (G_x - A_x)/A_w   t_y = (G_y - A_y)/A_h   t_w = log(G_w/A_w)   t_h =
log(G_h/A_h)
```

4.5 RPN's total loss

```
L = (1/N_cls) · sum(L_cls) + lambda · (1/N_reg) · sum(y_i · L_reg)
```

L_cls = classification loss (objectness: object vs background). L_reg = box regression loss, only computed for positive anchors (multiplied by y_i, since background anchors don't need a box refinement). lambda balances the two losses.

4.6 What projecting means here

RPN's predicted boxes are expressed in original image coordinates (that's the practical scale to describe an object's real size in). Before ROI pooling can use them, they go through the exact same projection step as Fast R-CNN — divide by the CNN's shrink factor — so the box correctly points to the matching small patch on the feature map. Everything after this (ROI pooling, FC layers, Softmax + box regressor) is identical to Fast R-CNN.

5. Summary — R-CNN family at a glance

	R-CNN	Fast R-CNN	Faster R-CNN
Region proposals	Selective search	Selective search	RPN (learned)
CNN runs	~2000x per image	1x per image	1x per image
Fixed-size step	Warp raw pixels	ROI pooling on feature map	ROI pooling on feature map
Classifier	Separate SVM	Softmax (joint)	Softmax (joint)
Box refine	Separate regressor	Joint, same network	Joint, same network (+ anchors)
Trainable end-to-end	No	Partly (SC not trainable)	Fully
Speed	Very slow	Faster, SC still bottleneck	Fastest, near real-time

Key formulas to remember

- Region similarity (selective search): s = a1·color + a2·texture + a3·size + a4·fill
- Box regression target: t_x=(G_x-P_x)/P_w, t_w=log(G_w/P_w) (same shape for y, h)
- IoU: Area(A∩G) / Area(A∪G)
- Objectness: p = sigmoid(z) = 1/(1+e^-z)
- Feature-map projection: coord_feat = coord_image / shrink_factor

Common pitfalls (my own mistakes while learning this)

- Thinking cropping happens only "when an object is found" — actually cropping/proposing happens first, blindly; classification happens after.
- Thinking selective search runs on the CNN's output — it actually runs independently on the raw image, in Fast R-CNN's two parallel paths.
- Thinking ROI pooling "resizes" like warping — it actually divides into a fixed grid and takes the max per cell; no pixel stretching involved.
- Thinking superpixels are a fixed grid (like 4x4 blocks) — they are irregular shapes that follow natural colour/texture boundaries.

Practice questions before moving on

- [Hand-calc] Given anchor A = (30,30,60,60) and ground truth G = (35,32,55,58), compute IoU and the regression targets t_x, t_y, t_w, t_h.
- [Hand-calc] For a 224x224 image with shrink factor 16, project the box (96,96,64,64) onto the feature map.

- [Concept] In your own words, explain why Faster R-CNN needs anchor boxes instead of letting the RPN predict box coordinates directly from scratch.
- [Code] Implement ROI pooling in NumPy for one feature map region of arbitrary size, pooled into a fixed 3x3 grid.
- [Code] Implement the objectness sigmoid + IoU-based anchor labelling for a small set of toy anchors and one ground-truth box.

End of R-CNN family notes. Next up: YOLO / SSD (single-shot detectors).